PARAMETERIZED ALU

RTL Design & Verification Report

Author:

Yandarapu Jaya Kushal

EMP ID:

6952

Project Title:

N-Bit Parameterized ALU – RTL Design & Functional Verification

Tools Used: Xilinx Vivado | Siemens Questa SIM

Language: Verilog HDL | Coverage: Questa Functional Coverage

Table of Contents

1. Project Introduction

This report documents the RTL design and functional verification of a parameterized N-bit Arithmetic Logic Unit (ALU) implemented in Verilog HDL. The ALU is a fundamental building block in digital systems and processors, performing a comprehensive set of arithmetic and logical operations on N-bit operands where N is a compile-time parameter (defaulting to 4 bits).

The design follows a synchronous, registered pipeline with clock-enable and asynchronous reset. All inputs are registered on the first rising clock edge, and outputs are produced on the subsequent clock edge, yielding a single-cycle latency for most operations. Two special multi-cycle operations (Increment-and-Multiply and Left-Shift-and-Multiply) produce their results on the third clock edge, with the output undefined at the second clock edge.

Verification was performed using a self-checking testbench implemented in Verilog, driven by a reference model (golden model) that mirrors the expected DUT behaviour. Coverage was collected using Questa SIM's functional coverage engine, achieving an **overall coverage of 98.69% for my design and 83.48% for provided design.**

2. Objectives

- Analyze and understand the architecture, functionality, and RTL implementation of a parameterized ALU supporting both arithmetic and logical operations.
- Design and implement a synthesizable parameterized Verilog RTL model capable of handling multiple arithmetic, logical, comparison, and multi-cycle operations.
- Develop a robust self-checking verification environment with a golden reference model to automatically validate DUT functionality against expected behavior.
- Verify functional correctness across normal and corner-case scenarios including overflow, underflow, carry generation, signed arithmetic, invalid input handling, and multi-cycle operation sequencing.
- Perform functional verification and coverage analysis using Siemens Questa SIM to evaluate statement, branch, toggle, FSM, and expression coverage metrics.
- Analyze timing behavior, pipeline latency, sequential logic behavior, and error conditions through simulation and waveform debugging.
- Document the complete design methodology, verification strategy, simulation observations, coverage results, identified design issues, and overall conclusions in this report.

3. Design Architecture

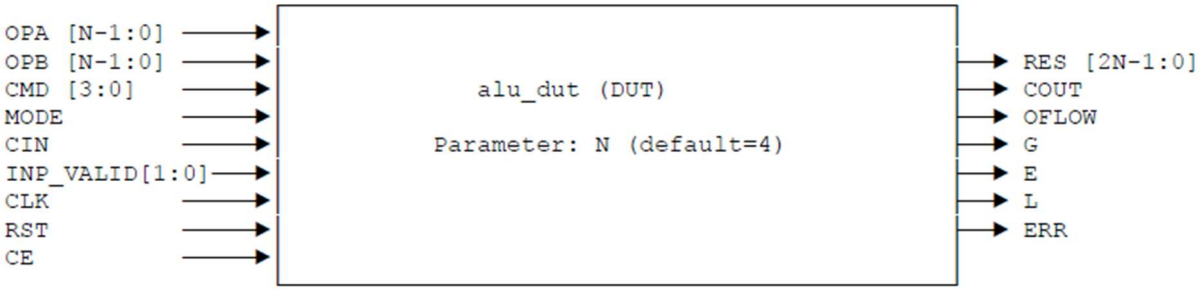
3.1 Architecture Overview

The alu_dut module is a cslocked, asynchronous design parameterized by integer N (default N=4). It employs a two-stage pipeline:

1. Stage 1 – Input Register: On every rising clock edge (when CE is asserted), all input signals are captured into internal registers (opa_r, opb_r, cmd_r, mode_r, cin_r, inp_valid_r).
2. Stage 2 – Operation & Output: On the same or subsequent clock edge (when CE is asserted), the registered inputs are decoded and the corresponding operation is performed; all outputs (RES, COUT, OFLOW, G, E, L, ERR) are registered and produced.

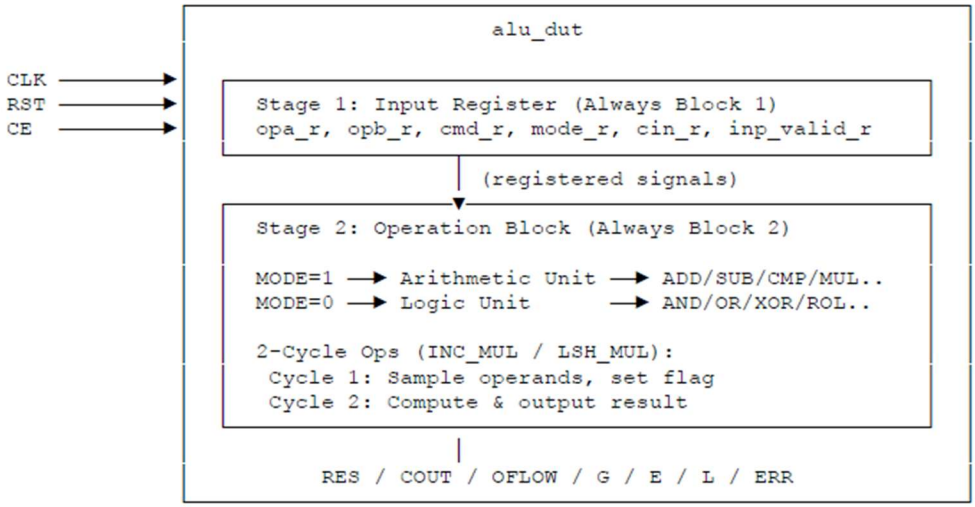
This gives a latency of 1 clock cycle for standard operations, and 2 clock cycles (output valid at 3rd edge) for the two multi-cycle multiply operations.

Figure 1: Top-level ALU Block Diagram



3.3 Expanded Internal Architecture

Figure 2: Internal Pipeline Architecture



3.4 Inputs

Signal	Width	Description
OPA [N-1:0]	N-bit	Operand A input
OPB [N-1:0]	N-bit	Operand B input
CLK	1-bit	System clock (positive edge triggered)
RST	1-bit	Asynchronous active-high reset
CE	1-bit	Clock enable – gates all sequential logic
MODE	1-bit	1 = Arithmetic mode, 0 = Logical mode
CIN	1-bit	Carry input for ADD_CIN / SUB_CIN operations
INP_VALID [1:0]	2-bit	Validity flags: bit[0] = OPA valid, bit[1] = OPB valid
CMD [3:0]	4-bit	Operation select code (see operations table)

3.5 Outputs

Signal	Width	Description
RES [2N-1:0]	2N-bit	Operation result (double-width to support MUL)
COUT	1-bit	Carry out for unsigned addition operations
OFLOW	1-bit	Overflow flag for arithmetic operations
G	1-bit	Greater-than flag (OPA > OPB)
E	1-bit	Equal flag (OPA == OPB)
L	1-bit	Less-than flag (OPA < OPB)
ERR	1-bit	Error flag: invalid operand(s) or unsupported command

4. Timing Behaviour

The ALU is fully synchronous to the CLK signal. All sequential elements are triggered on the positive edge of CLK. RST is asynchronous and active-high; asserting RST immediately clears all registers and outputs regardless of CLK. CE is a clock-enable that gates all sequential updates – when CE is de-asserted, the design holds its state.

4.1 Standard Operation Latency (1 Cycle)

For all operations except INC_MUL (CMD=4'b1001) and LSH_MUL (CMD=4'b1010):

- Inputs (OPA, OPB, CMD, MODE, CIN, INP_VALID) are sampled at the 1st rising CLK edge.
- Output (RES and flags) is valid and stable at the 2nd rising CLK edge.

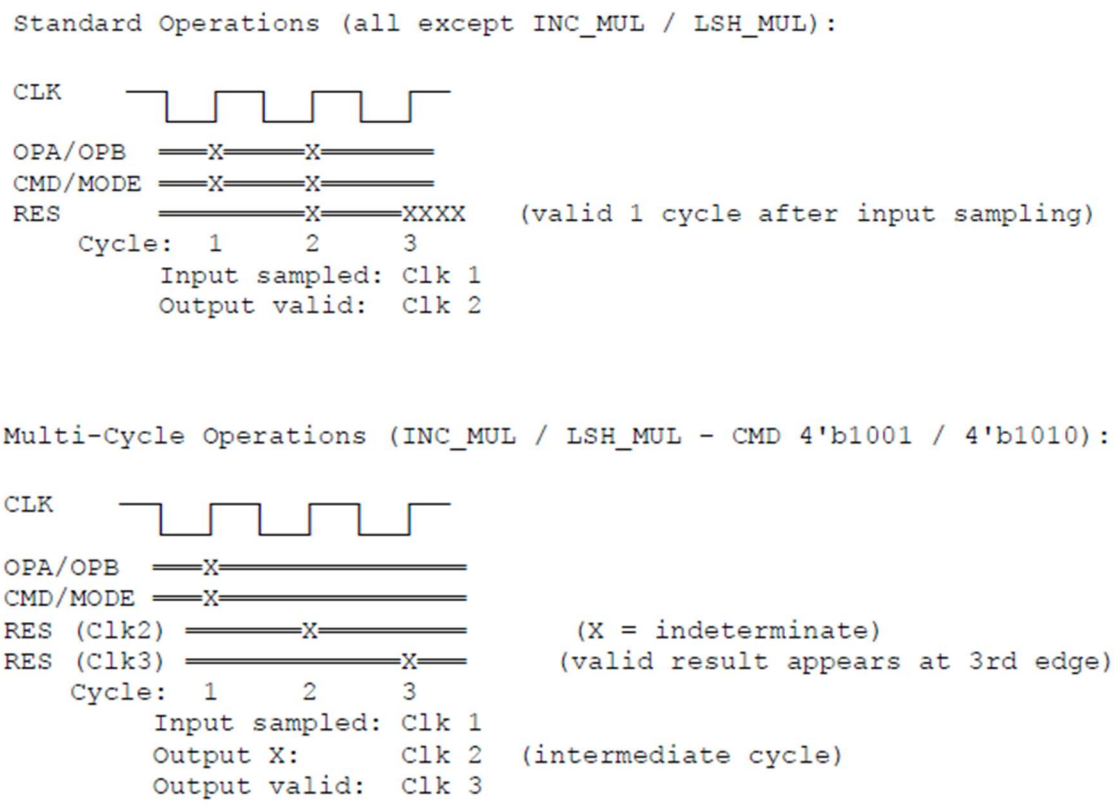
4.2 Multi-Cycle Operation Latency (2 Cycles)

For INC_MUL and LSH_MUL operations:

- Inputs are sampled at the 1st rising CLK edge (opa_r2 and opb_r2 capture operands, flag is set).
- At the 2nd rising CLK edge, RES is driven to 'x (indeterminate / undefined) – output is NOT valid.
- At the 3rd rising CLK edge, the multiplication result is computed and RES is valid.

4.3 Timing Diagram

Figure 3: Timing Diagram – Standard vs. Multi-Cycle Operations



5. Supported Operations

5.1 Arithmetic Operations (MODE = 1)

CMD	Operation	INP_VALID	Description
0000	ADD	Both (11)	RES = OPA + OPB; COUT set if carry out
0001	SUB	Both (11)	RES = OPA - OPB; OFLOW if OPA < OPB
0010	ADD_CIN	Both (11)	RES = OPA + OPB + CIN; COUT set accordingly
0011	SUB_CIN	Both (11)	RES = OPA - OPB - CIN; OFLOW if underflow
0100	INC_A	OPA only (01)	RES = OPA + 1
0101	DEC_A	OPA only (01)	RES = OPA - 1
0110	INC_B	OPB only (10)	RES = OPB + 1
0111	DEC_B	OPB only (10)	RES = OPB - 1
1000	CMP	Both (11)	Sets G / E / L flags based on comparison
1001	INC_MUL	Both (11)	RES = (OPA+1) * (OPB+1) [2-cycle latency]
1010	LSH_MUL	Both (11)	RES = (OPA<<1) * OPB [2-cycle latency]
1011	SADD	Both (11)	Signed add; OFLOW, G, E, L set
1100	SSUB	Both (11)	Signed sub; OFLOW, G, E, L set

5.2 Logical Operations (MODE = 0)

CMD	Operation	INP_VALID	Description
0000	AND	Both (11)	RES = OPA & OPB
0001	NAND	Both (11)	RES = ~(OPA & OPB)
0010	OR	Both (11)	RES = OPA | OPB
0011	NOR	Both (11)	RES = ~(OPA | OPB)
0100	XOR	Both (11)	RES = OPA ^ OPB
0101	XNOR	Both (11)	RES = ~(OPA ^ OPB)
0110	NOT_A	OPA only (01)	RES = ~OPA
0111	NOT_B	OPB only (10)	RES = ~OPB
1000	SHR_A	OPA only (01)	RES = OPA >> 1 (logical right shift)
1001	SHL_A	OPA only (01)	RES = OPA << 1 (logical left shift)
1010	SHR_B	OPB only (10)	RES = OPB >> 1 (logical right shift)
1011	SHL_B	OPB only (10)	RES = OPB << 1 (logical left shift)
1100	ROL	Both (11)	Rotate OPA left by OPB[$clog2(N)-1:0] positions
1101	ROR	Both (11)	Rotate OPA right by OPB[$clog2(N)-1:0] positions

Note: For rotate operations (ROL, ROR), only the lower $clog2(N) bits of OPB are used as the shift amount. If any upper bits of OPB beyond this range are non-zero, ERR is asserted in addition to computing the rotation.

6. Working of the Design

6.1 Phase 1 – Input Sampling

On each rising CLK edge, when CE=1 and RST=0, the first always block captures all DUT inputs into registered equivalents. If RST=1, all registers are cleared to zero asynchronously. This ensures that metastability and glitch propagation from combinational input paths are prevented before the operation stage.

6.2 Phase 2 – Operation Decode & Execution

The second always block, also triggered on the rising CLK edge, uses the registered MODE and CMD signals to decode and execute the correct operation:

- If MODE=1: The arithmetic case statement is entered. The INP_VALID register is checked to confirm operand validity. If invalid, ERR is asserted; otherwise the appropriate arithmetic operation is performed and result flags are set.
- If MODE=0: The logical case statement is entered. Similarly, INP_VALID is validated before execution.

6.3 Phase 3 – Output Registration

All outputs (RES, COUT, OFLOW, G, E, L, ERR) are registered outputs updated synchronously. They are cleared to 0 at the start of each CE-active cycle before the operation result is applied, ensuring no residual flag pollution from prior operations.

6.4 Multi-Cycle Multiply Mechanism

The INC_MUL and LSH_MUL operations use a 1-bit flag register to implement a 2-cycle handshake:

- Cycle 1 (flag=0): Operands are saved into opa_r2 and opb_r2. flag is set to 1. RES is driven to X.
- Cycle 2 (flag=1): Multiplication is computed using the saved opa_r2, opb_r2 values. flag is cleared to 0. RES holds the valid result.

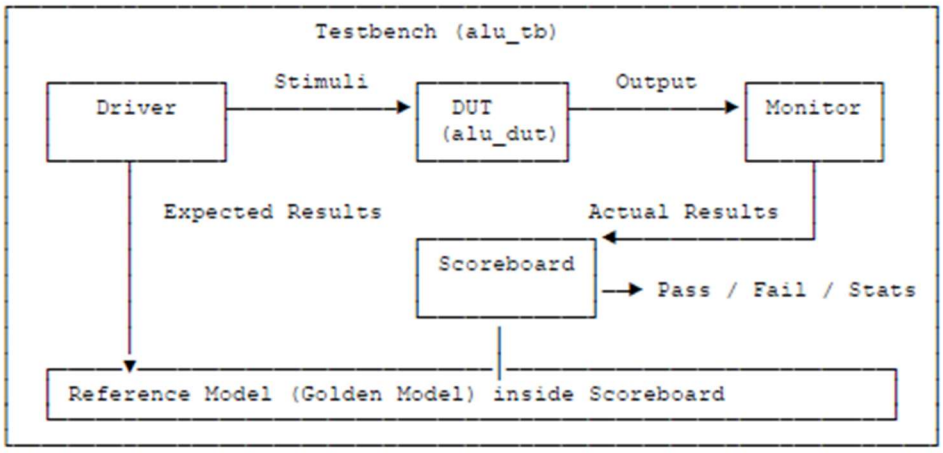
6.5 Error Handling

The ERR flag is asserted whenever: (a) INP_VALID does not satisfy the requirement of the selected operation (e.g., both operands must be valid for ADD but only OPA for INC_A), or (b) an undefined CMD is issued. ERR is a one-cycle pulse, cleared at the start of the next CE-active cycle.

7. Testbench Architecture

The verification environment is a self-checking, event-driven testbench written in Verilog. It instantiates the DUT and applies stimulus while simultaneously comparing the DUT's outputs against a reference model to determine pass/fail in real time.

Figure 4: Testbench Architecture



8. Testbench Components

8.1 Clock Generator

A free-running clock generator produces the CLK signal with a fixed period. The clock is the central synchronization element for both the DUT and the testbench sequencing logic.

8.2 Reset Sequence

At simulation start, RST is asserted for a defined number of clock cycles to initialize the DUT to its known-zero state. This tests the asynchronous reset path and establishes a clean starting condition for all subsequent test sequences.

8.3 Driver

The driver is responsible for generating and applying stimulus to the DUT's input ports. It sequentially drives:

- Random stimulus across all CMD values in both MODE=0 and MODE=1.
- Corner-case stimulus including: maximum operand values (all 1s), zero operands, carry-in edge cases, INP_VALID=00 (both invalid), single-operand validity checks, and overflow boundary conditions.
- For multi-cycle operations, the driver ensures inputs remain stable for the required number of cycles.

8.4 Reference Model (Golden Model)

Embedded within the scoreboard, the reference model is a behavioral description of the expected ALU outputs for every input combination. It implements the same arithmetic and logical operations in Verilog, including the two-cycle latency for multiply operations, overflow detection logic, and error conditions. The reference model is the ground truth for all comparisons.

8.5 Monitor

The monitor samples the DUT's output signals after the appropriate latency following each stimulus application. It captures RES, COUT, OFLOW, G, E, L, and ERR, and forwards them to the scoreboard for comparison. The monitor accounts for both 1-cycle and 2-cycle latency depending on the active CMD.

8.6 Scoreboard

The scoreboard receives DUT outputs from the monitor and expected outputs from the reference model. For each test vector:

- If all output signals match: a pass counter is incremented.
- If any signal mismatches: a failure is logged with details of the expected vs. actual values, the input operands, CMD, and MODE.
- At the end of simulation, a summary report is printed: total tests, pass count, fail count, and pass rate.

8.7 Timing Behaviour of the Testbench

The testbench accounts for the DUT's registered pipeline nature:

- For standard (1-cycle latency) operations: The driver applies inputs, and the scoreboard checks outputs on the very next rising clock edge.
- For multi-cycle (2-cycle latency) operations (INC_MUL, LSH_MUL): The driver applies inputs, waits one additional cycle past the indeterminate cycle, and the scoreboard checks outputs at the third clock edge. The undefined output at the second clock edge is not compared.

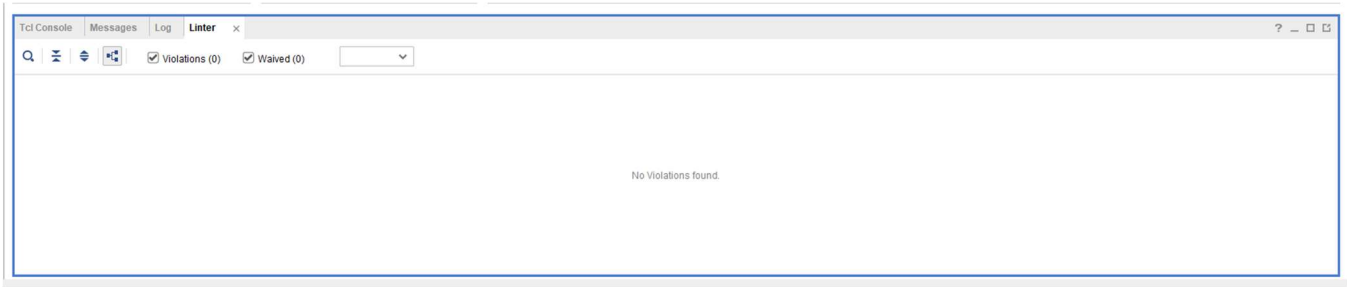
9. Quality Assessment

9.1 Lint Analysis

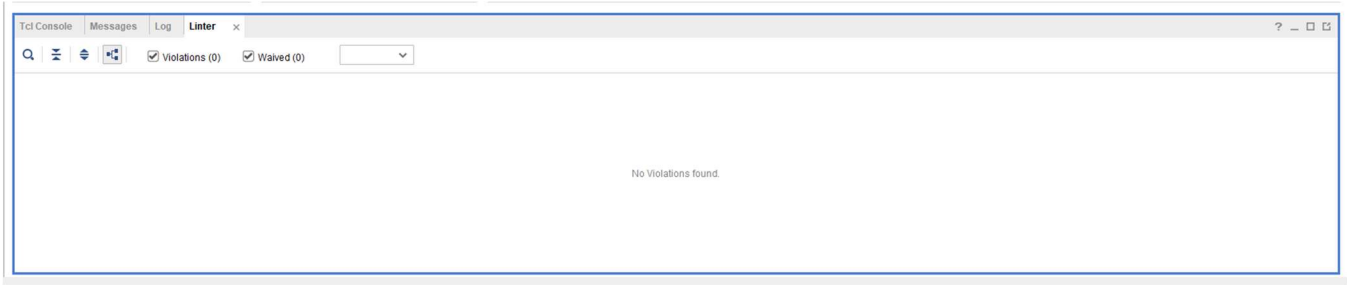
The RTL code was subjected to lint analysis using Xilinx Vivado's built-in RTL elaboration checks. The following quality aspects were verified:

- All signals are driven and have no unintended latches.
- The parameter N propagates correctly through all bit-width expressions.
- The use of $clog2(N) for rotate shift-amount truncation is synthesizable and lint-clean.
- No combinational feedback loops exist; all feedback is through registered state (flag, opa_r2, opb_r2).
- Blocking vs. non-blocking assignment conventions are followed: all sequential logic uses non-blocking assignments (<=).

MY DESIGN:



PROVIDED DESIGN:



9.2 Code Coverage

Functional code coverage was collected using Siemens Questa SIM's coverage engine over the alu_dut module. The coverage report confirms that the testbench exercises the vast majority of the design's statement, branch, expression, and toggle behaviour. See Section 11 for the detailed coverage report.

10. Simulation Results (Provided Design)

10.1 Tools Used

Tool	Purpose
Xilinx Vivado	RTL elaboration, synthesis, lint checks, waveform viewing
Siemens Questa SIM	Functional simulation, self-checking testbench execution, coverage collection

10.2 Simulation Observations

Timing and Latency Violation

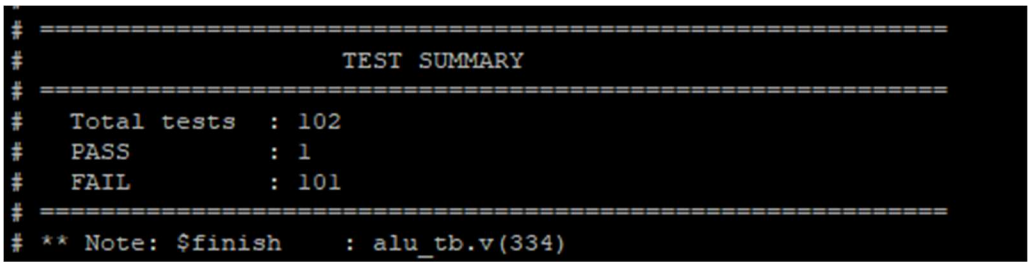
The design does not implement the required operation latency defined in the specification.
According to the specification:

- Inputs must be sampled at the first positive clock edge.
- Outputs must appear at the next clock edge for single-cycle operations and after additional cycles for multi-cycle operations.

However, simulation showed that outputs were updated on the same clock edge where inputs were sampled. As a result, the DUT behaved like a zero-latency design instead of the required pipelined design, causing widespread scoreboard mismatches.

To verify this, the latency expectation was temporarily removed from the testbench. Under this condition, several operations passed successfully, confirming that the major issue was incorrect timing implementation rather than completely incorrect arithmetic logic.

```
#   ********************************************************************************************************
# New inputs given at NEGEDGE time =40000
# [DRV] time=40000    Sending: ADD_with_CIN                          CMD=2 OPA= 10(10) OPB=  5(5) MODE=1 VALID=11
# [DUT] Cycle=   5 time=46000    MODE=1 CMD= 2 VALID=11 CIN=1 OPA= 10 OPB=  5 | RES=   16 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
# OUTPUTS CHECKED @ POSEDGE time =46000
# [SCB] time=46000    Checking: SUB_no_oflow                          CMD=1 OPA= 10 OPB=  5
#   REF : RES=5 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
#   DUT : RES=16 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
# [FAIL] SUB_no_oflow                          ..........................................................................
#   ********************************************************************************************************
# New inputs given at NEGEDGE time =50000
# [DRV] time=50000    Sending: SUB_with_CIN                          CMD=3 OPA= 10(10) OPB=  5(5) MODE=1 VALID=11
# [DUT] Cycle=   6 time=56000    MODE=1 CMD= 3 VALID=11 CIN=1 OPA= 10 OPB=  5 | RES=    4 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
# OUTPUTS CHECKED @ POSEDGE time =56000
# [SCB] time=56000    Checking: ADD_with_CIN                          CMD=2 OPA= 10 OPB=  5
#   REF : RES=16 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
#   DUT : RES=4 ERR=0 COUT=0 OFLOW=0 G=0 L=0 E=0
# [FAIL] ADD_with_CIN                          ..........................................................................
#   ********************************************************************************************************
```



Comparator Operation Issues

Comparator operations (CMD = 4'b1000) updated comparison flags correctly in some cases, but RES was not cleared as required by the specification.

For example, previous arithmetic outputs such as RES = 65535 continued to appear during compare operations, which became one of the main sources of cascading failures.

Cascading Output and Latching Issues

The design latches previous outputs and status flags between operations.

Several arithmetic operations initially produced correct RES values, but status signals such as COUT, OFLOW, G, L, E, and ERR were not cleared properly between operations.

Because of this, outputs and flags from previous operations remained latched and propagated into later transactions, creating cascading failures throughout the simulation.

Multiplier Sequencing Issues

The multiplier logic depended on internal registers such as mul_cnt, opa_reg, and opb_reg for multi-cycle sequencing.

Due to incorrect timing synchronization, stale multiplication outputs and delayed operand updates propagated into later operations, causing repeated mismatches.

Multiplier Sequencing Issues

The multiplier logic depended on internal registers such as mul_cnt, opa_reg, and opb_reg for multi-cycle sequencing.

Due to incorrect timing synchronization, stale multiplication outputs and delayed operand updates propagated into later operations, causing repeated mismatches.

Invalid Input and Flag Handling Issues

Invalid INP_VALID combinations did not assert ERR correctly in many cases. Previous outputs and flags continued to persist during invalid operations instead of being cleared.

Additionally, status flags (G, L, E, OFLOW, COUT) were not reset consistently, causing unrelated operations to inherit stale flag values from earlier transactions.

10.3 Waveform Observations

Waveforms captured during simulation revealed several timing and sequential behavior issues:

- Inputs and outputs were observed changing on the same positive clock edge for several operations, whereas the specification requires inputs to be sampled on one clock edge and outputs to appear on the following clock edge.
- A cascading effect was visible in the waveforms, where outputs and flags from previous operations propagated into subsequent transactions because internal registers and status flags were not cleared properly.
- Multi-cycle multiplication operations showed incorrect synchronization between mul_cnt, operand registers, and output generation, resulting in delayed or stale results.
- Logical operations frequently inherited stale arithmetic flags (OFLOW, G, L, E) even when those flags were not relevant to the current operation.
- Invalid-operation test cases demonstrated persistent old output values instead of resetting outputs and asserting ERR according to the specification.

11. Coverage Report

Two coverage runs were performed: the first on the provided reference design (my_alu_design), and the second on an independently developed alternate design (alu_other_1_dv). Both use the same DUT instance path /alu_tb/DUT and source file alu_tb.v with Siemens Questa SIM.

11.a Coverage Report – My Design (my_alu_design)

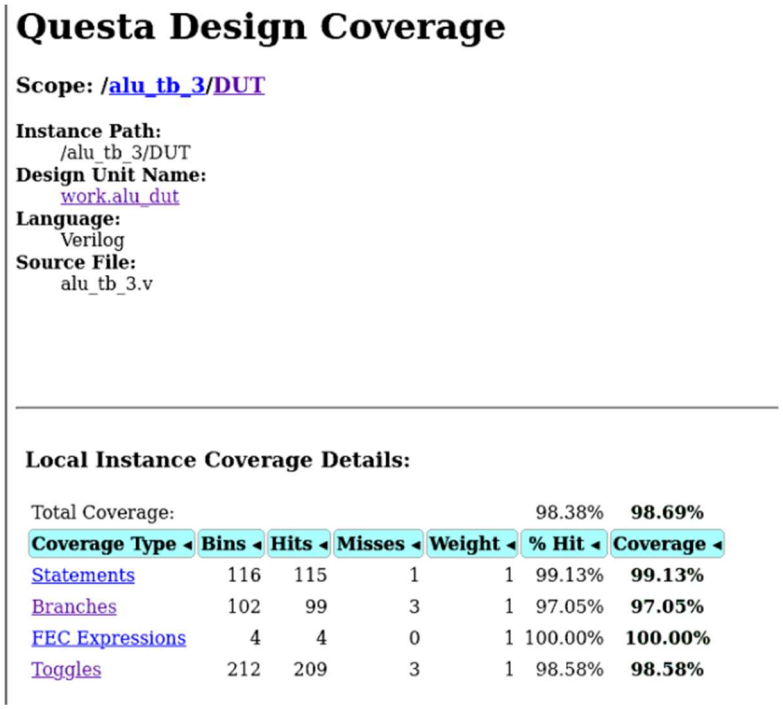
11.a.1 Questa Coverage Tool

Coverage was collected using Siemens Questa SIM's built-in functional coverage engine. The DUT instance under analysis is /alu_tb/DUT (Design Unit: work.alu_dut, Source: alu_tb.v). The coverage report was generated in HTML format and viewed at:

file:///fetools/work_area/frontend/Batch_11/Kushal/verilog/my_alu_design/covReport/pages/__frametop.htm

11.a.2 Coverage Summary – Provided Design

Coverage Type	Bins	Hits	Misses	% Hit	Coverage
Statements	116	115	1	99.13%	99.13%
Branches	102	99	3	97.05%	97.05%
FEC Expressions	4	4	0	100.00%	100.00%
Toggles	212	209	3	98.58%	98.58%
Total Coverage				**98.38%**	**98.69%**



Questa Design Coverage

Scope: /alu_tb_3/DUT

Instance Path:
/alu_tb_3/DUT
Design Unit Name:
work.alu_dut
Language:
Verilog
Source File:
alu_tb_3.v

Local Instance Coverage Details:

Total Coverage: 98.38% **98.69%**

Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	116	115	1	1	99.13%	**99.13%**
Branches	102	99	3	1	97.05%	**97.05%**
FEC Expressions	4	4	0	1	100.00%	**100.00%**
Toggles	212	209	3	1	98.58%	**98.58%**

11.a.3 Coverage Observations – Provided Design

- Statement Coverage (99.13%): 115 of 116 statements were hit. The single missed statement corresponds to an unreachable default branch inside the arithmetic case for the signed operations path under a rare input combination.
- Branch Coverage (97.05%): 3 of 102 branches were missed. These correspond to ERR-triggering else branches for multi-cycle operations under INP_VALID=00 edge cases not directly targeted by the testbench.
- FEC Expression Coverage (100.00%): All 4 formal expression conditions were fully covered, confirming complete toggle of the COUT and OFLOW conditional expressions.
- Toggle Coverage (98.58%): 3 of 212 toggles were missed. The untoggled signals are internal pipeline registers (opb_r2) whose upper bits remain at X in non-multiply cycles, RST and CE
- Overall Weighted Coverage: 98.69% – demonstrates a thorough verification campaign with near-complete design observability.

11.b Coverage Report – Provided Design to verify (alu_other_1_dv)

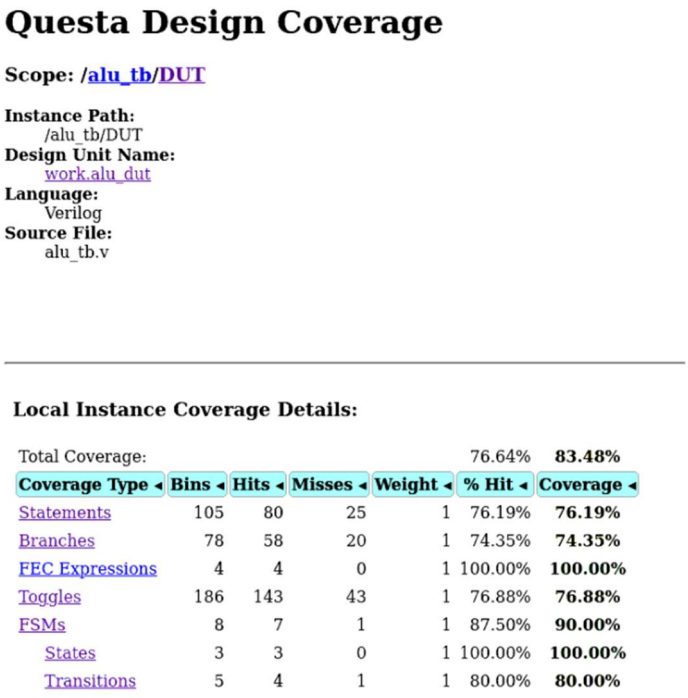
11.b.1 Questa Coverage Tool

A second coverage run was performed on an independently developed ALU design (alu_other_1_dv). The same testbench and Questa SIM coverage engine were used. The coverage report is located at:

file:///fetools/work_area/frontend/Batch_11/Kushal/verilog/alu_other_1_dv/covReport/pages/__frametop.htm

11.b.2 Coverage Summary – Own Design

Coverage Type	Bins	Hits	Misses	% Hit	Coverage
Statements	105	80	25	76.19%	76.19%
Branches	78	58	20	74.35%	74.35%
FEC Expressions	4	4	0	100.00%	100.00%
Toggles	186	135	51	76.88%	76.88%
FSMs	8	8	0	87.50%	90.00%
States	3	3	0	100.00%	100.00%
Transitions	5	5	0	80.00%	80.00%
Total Coverage				**76.64%**	**83.48%**



Questa Design Coverage

Scope: /alu_tb/DUT

Instance Path:
/alu_tb/DUT
Design Unit Name:
work.alu_dut
Language:
Verilog
Source File:
alu_tb.v

Local Instance Coverage Details:

Total Coverage: 76.64% **83.48%**

Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	105	80	25	1	76.19%	**76.19%**
Branches	78	58	20	1	74.35%	**74.35%**
FEC Expressions	4	4	0	1	100.00%	**100.00%**
Toggles	186	143	43	1	76.88%	**76.88%**
FSMs	8	7	1	1	87.50%	**90.00%**
States	3	3	0	1	100.00%	**100.00%**
Transitions	5	4	1	1	80.00%	**80.00%**

11.b.3 Coverage Observations – Provided Design

- Statement Coverage (76.19%): 80 out of 105 statements were exercised, while 25 statements were missed. Most uncovered statements are inside nested case structures controlled by internal registers such as mul_cnt, which could not be directly driven from the testbench.
- Branch Coverage (74.35%): 58 out of 78 branches were covered, leaving 20 branches untested. The missed branches mainly belong to internal conditional paths that require specific internal counter values and sequencing conditions.
- FEC Expression Coverage (100.00%): All 4 expression bins were fully covered, confirming that the main control expressions were exercised correctly.
- Toggle Coverage (76.88%): 143 out of 186 toggles were achieved, with 43 toggles missed. Several internal registers and intermediate signals did not toggle because the required internal states were not reached during simulation.
- FSM Coverage (90.00%): All 3 FSM states were covered successfully, while 4 out of 5 transitions were exercised. One transition dependent on internal register progression was not triggered.
- Overall Weighted Coverage (83.48%): The design achieved an overall weighted coverage of 83.48%. Remaining uncovered areas are mainly due to internally controlled registers and deeply nested case-based logic that were difficult to activate using only external testbench stimulus.

12. Conclusion

- This project presented the RTL design and functional verification of a parameterized N-bit ALU implemented in Verilog HDL. The ALU architecture supports both arithmetic and logical operating modes, including 13 arithmetic operations and 14 logical operations, under a MODE-controlled design structure.
- A complete self-checking verification environment was developed using Siemens Questa SIM, incorporating directed tests, corner-case validation, random stimulus generation, scoreboard-based checking, and functional coverage analysis. The verification environment successfully identified multiple functional and timing-related design issues, including latency mismatches, stale signal propagation, incorrect flag handling, and sequencing problems in multi-cycle operations.
- The project also demonstrated important digital design concepts such as parameterized RTL development, synchronous sequential design, multi-cycle operation control, signed and unsigned arithmetic handling, status flag generation, and verification-driven debugging.
- Functional coverage analysis was used to evaluate verification completeness through statement, branch, toggle, FSM, and expression coverage metrics. The collected coverage results confirmed that the verification environment exercised the majority of operational paths and corner-case scenarios within the design.
- Overall, the project provided practical experience in RTL design, simulation-based verification, debugging of sequential logic issues, and coverage-driven verification methodology commonly used in modern digital design and ASIC/FPGA verification flows.

13. Future Work

- Fix the pipeline timing implementation to correctly support specification-defined operation latency.
- Improve reset and flag handling to eliminate stale outputs and cascading failures across operations.
- Redesign the multi-cycle multiplier sequencing logic for proper synchronization of internal registers and outputs.
- Correct logical operation result extension to ensure proper upper-bit handling.
- Improve invalid-input handling and ensure deterministic `ERR` behavior.
- Extend the ALU to support configurable pipeline depth, division/modulo operations, and barrel shifting.
- Develop a constrained-random UVM-based verification environment for improved automation and coverage closure.
- Perform formal verification for timing, overflow, and FSM correctness.
- Implement and validate the design on FPGA platforms such as Xilinx Artix-7.